Phần 1: Phân tích & So sánh

Tiêu chí	Cách 1: Đường dẫn tuyệt đối (`Absolute Path`)	Cách 2: Đường dẫn tương đối (`Relative Path`)
	`C:\Users\An\Desktop\Project\Data\users.csv`	`../Data/users.csv`
Tính di động (Portability)	**Rất kém**: Bị "hard-code" gắn chặt với máy tính của An (ổ đĩa `C:`, tên tài khoản `An`). Khi gửi thư mục cho bạn bè hoặc chạy trên môi trường khác (Linux, Docker, máy chủ), code lập tức báo lỗi `FileNotFoundError`.	**Rất cao**: Độc lập với thư mục gốc của hệ điều hành. Miễn là cấu trúc nội bộ giữa thư mục `src` và thư mục `Data` giữ nguyên, project chuyển sang máy khác, ổ đĩa khác hay hệ điều hành khác đều chạy được.
Độ dài câu lệnh	Dài dòng, cồng kềnh, khó bảo trì nếu cấu trúc thư mục cha bên ngoài thay đổi vị trí lưu trữ.	Ngắn gọn, súc tích, dễ nhìn và dễ quản lý trong code.
Tính an toàn & Ổn định	**An toàn ngữ cảnh**: Luôn trỏ tới đúng một vị trí vật lý cố định trên ổ cứng, không phụ thuộc vào thư mục hiện hành khi gõ lệnh chạy (`Current Working Directory - CWD`).	**Rủi ro CWD**: Phụ thuộc hoàn toàn vào vị trí bạn đứng khi chạy lệnh terminal. Nếu đứng ở thư mục gốc `Project/` thay vì `src/` để gõ lệnh chạy, đường dẫn `../Data/...` sẽ bị trỏ sai vị trí.

Phần 2: Lựa chọn giải pháp tối ưu

1. Đề xuất lựa chọn

Theo quy tắc nghiệp vụ (yêu cầu dự án chạy được trên máy của người khác), **Cách 2 (Đường dẫn tương đối)** là giải pháp bắt buộc phải dùng để đảm bảo tính di động (**Portability**).

2. Ý nghĩa của các ký hiệu `.` và `..`

- `.` (Dấu chấm đơn): Đại diện cho **thư mục hiện hành** (Current Directory) — nơi tiến trình hoặc tệp mã nguồn đang đứng.
- `..` (Dấu hai chấm): Đại diện cho **thư mục cha** (Parent Directory) — lùi lên một cấp thư mục so với vị trí hiện tại.
- **Giải thích ngữ cảnh của An**: Mã nguồn đặt tại thư mục `Project/src/`. Ký hiệu `../Data/users.csv` có nghĩa:
  - `..` : Thoát khỏi thư mục con `src` để lùi về thư mục cha `Project`.
  - `Data/users.csv` : Đi vào thư mục con `Data` nằm trong `Project` và trỏ tới tệp `users.csv`.

Mẹo chuyên nghiệp (Best Practice trong Python)

Để khắc phục nhược điểm phụ thuộc vào CWD (vị trí đứng gõ lệnh terminal) của Cách 2, lập trình viên chuyên nghiệp sẽ kết hợp đường dẫn tương đối dựa trên vị trí của chính file script (`__file__`) thông qua thư viện `pathlib`:

Python

```
from pathlib import Path

# Xác định đường dẫn tuyệt đối động dựa trên vị trí file mã nguồn hiện tại
BASE_DIR = Path(__file__).resolve().parent  # Trỏ tới thư mục Project/src
DATA_FILE = BASE_DIR.parent / "Data" / "users.csv"  # Lùi 1 cấp về Project, vào Data

# DATA_FILE sẽ tự động nhận diện đúng file trên mọi máy tính và hệ điều hành (Windows, Linux, macOS)
```

Cách tiếp cận này vừa đạt 100% tính di động khi chia sẻ cho bạn bè, vừa tránh lỗi đường dẫn khi chạy script từ bất kỳ thư mục nào.